



Gestor de gastos: filtrado y cálculo de agrupaciones de gastos

Este caso práctico es continuación del anterior, por lo que seguiremos utilizando el mismo repositorio de código. Para esta práctica, los test se ejecutan con `npm run test3`. De nuevo, si se está utilizando Visual Studio Code se pueden ejecutar los test desde el propio editor.

Utilizaremos de nuevo el fichero `js/gestionPresupuesto.js`. A no ser que se indique lo contrario, todo el código que se cree deberá guardarse en este fichero.

Nuestra librería ha crecido en prestaciones: ya nos permite crear gastos con información variada (nombre, cantidad, fecha y etiquetas) y gestionar el listado de ellos.

En esta práctica dotaremos a nuestra librería de algunas funciones muy útiles, como son el **filtrado para realizar búsquedas** y la agrupación de datos para **calcular totales por periodos temporales** (gastos mensuales, por ejemplo).

Las tareas que hay que realizar son las siguientes:

- Añade las funciones `filtrarGastos` y `agruparGastos` al objeto `export` del final del fichero. Define las funciones vacías (sin parámetros y sin cuerpo) en el fichero `js/gestionPresupuesto.js`, para que los test no den error de sintaxis y se puedan ir comprobando conforme se vaya avanzando en la práctica.
- Actualiza la función constructora `CrearGasto` para que los objetos `gasto` tengan acceso a una nueva función llamada `obtenerPeriodoAgrupacion`. Será una función de un parámetro, que devolverá el periodo de agrupación correspondiente al parámetro `periodo` de la función y a la fecha del gasto. Si el periodo que se debe agrupar es `dia`, el periodo de agrupación tendrá el formato `aaaa-mm-dd`; si es `mes`, tendrá el formato `aaaa-mm`; y si es `año`, tendrá el formato `aaaa`. A continuación, se muestran algunos ejemplos:

```
let gasto1 = new CrearGasto("Gasto 1", 23.55, "2021-09-06", "casa",  
  "supermercado" );  
  
let gasto2 = new CrearGasto("Gasto 2", 27.55, "2021-11-24", "casa",  
  "supermercado", "comida" );
```

```
gasto1.obtenerPeriodoAgrupacion("mes");  
// Resultado: "2021-09"  
  
gasto1.obtenerPeriodoAgrupacion("año");  
// Resultado: "2021"  
  
gasto1.obtenerPeriodoAgrupacion("dia");  
// Resultado: "2021-09-06"
```

```
gasto2.obtenerPeriodoAgrupacion("mes");  
// Resultado: "2021-11"  
  
gasto2.obtenerPeriodoAgrupacion("año");  
// Resultado: "2021"  
  
gasto2.obtenerPeriodoAgrupacion("dia");  
// Resultado: "2021-11-24"
```

- Crea una función **filtrarGastos**. Será una función de un parámetro, que devolverá un subconjunto de los gastos existentes (variable global **gastos**). Se deberá utilizar la función **filter**. El parámetro será un objeto que podrá tener las siguientes propiedades:
 - **fechaDesde**. Fecha mínima de creación del gasto. Su valor deberá ser un **string** con formato válido que pueda entender la función **Date.parse**.
 - **fechaHasta**. Fecha máxima de creación del gasto. Su valor deberá ser un **string** con formato válido que pueda entender la función **Date.parse**.
 - **valorMinimo**. Valor mínimo del gasto.
 - **valorMaximo**. Valor máximo del gasto.
 - **descripcionContiene**. Trozo de texto que deberá aparecer en la descripción. Deberá hacerse la comparación de manera que no se distingan mayúsculas de minúsculas.
 - **etiquetasTiene**. Array de etiquetas: si un gasto contiene alguna de las etiquetas indicadas en este parámetro, se deberá devolver en el resultado. Deberá hacerse la comparación de manera que no se distingan mayúsculas de minúsculas.
- Algunos ejemplos de llamadas de función **filtrarGastos** podrían ser:


```
filtrarGastos({});
filtrarGastos({fechaDesde: "2021-10-10"});
filtrarGastos({fechaDesde: "2021-10-10", fechaHasta: "2021-10-15"});
filtrarGastos({valorMinimo: 10});
filtrarGastos({valorMinimo: 10, valorMaximo: 50});
filtrarGastos({fechaDesde: "2021-10-10", fechaHasta: "2021-10-15", valorMaximo: 100});
filtrarGastos({descripcionContiene: "carne", valorMinimo: 10, valorMaximo: 50});
filtrarGastos({valorMaximo: 50, etiquetasTiene: ["alimentacion"]});
filtrarGastos({etiquetasTiene: ["alimentacion", "gasolina"]});
filtrarGastos({etiquetasTiene: ["alimentacion", "gasolina"], fechaDesde: "2021-10-10"});
filtrarGastos({etiquetasTiene: ["alimentacion", "gasolina"], fechaHasta: "2020-12-31", valorMaximo: 200});
```
- Crea una función **agruparGastos**. Será una función de cuatro parámetros, que devolverá un objeto con los resultados de realizar una agrupación por periodo temporal. Los parámetros son:
 - **periodo**. Periodo utilizado para hacer la agrupación. Podrá ser uno de estos tres valores: **dia**, **mes** y **anyo**. El valor por defecto será **mes**.
 - **etiquetas**. Array de etiquetas. Solo se seleccionarán los gastos que contengan alguna de esas etiquetas. Si no se indica, o es un array vacío, se considerarán todos los gastos.
 - **fechaDesde**. Fecha mínima de creación del gasto. Su valor deberá ser un **string** con formato válido que pueda entender la función **Date.parse**. Si no se indica, se considerarán todos los gastos independientemente de su fecha.
 - **fechaHasta**. Fecha máxima de creación del gasto. Su valor deberá ser un **string** con formato válido que pueda entender la función **Date.parse**. Si no se indica, se considerará la **fecha actual**.



- Algunos ejemplos de resultados de ejecución de `agruparGastos` podrían ser:

```
gastos = [
  {id: 0, descripcion: "Gasto 1", valor: 5, fecha: "2021-09-30",
  etiquetas: ["alimentacion"]},
  {id: 1, descripcion: "Gasto 1", valor: 10, fecha: "2021-10-01",
  etiquetas: ["alimentacion"]},
  {id: 2, descripcion: "Gasto 1", valor: 12, fecha: "2021-10-02",
  etiquetas: ["transporte"]},
  {id: 3, descripcion: "Gasto 1", valor: 17, fecha: "2021-10-02",
  etiquetas: ["alimentacion"]}
];

// Suponemos fecha actual 2021-10-15

let agrup1 = agruparGastos("mes");
/*agrup1 = {
  "2021-09": 5,
  "2021-10": 39
}*/

let agrup2 = agruparGastos("dia");
/*agrup2 = {
  "2021-09-30": 5,
  "2021-10-01": 10,
  "2021-10-02": 29
}*/

let agrup3 = agruparGastos("mes", ["alimentacion"]);
/*agrup3 = {
  "2021-09": 5,
  "2021-10": 27
}*/
```

Claves de resolución

En primer lugar, comprueba que pasas los test de la práctica anterior. Si no es el caso, completa esa parte primero, antes de proseguir.

Dedica un tiempo a familiarizarte con el enunciado y todas las tareas que se piden. Inspecciona también el archivo del segundo test, para ver cómo funciona.

Por último, empieza a escribir el código que se pide en el enunciado. Cada vez que creas haber completado una de las tareas que se piden, lanza los test mediante `npm run test3` y comprueba si superas alguno nuevo. Ve realizando *commits* en el repositorio de manera periódica; sobre todo, cuando consigas superar un nuevo test.

Si tienes problemas con la función `agruparGastos`, puedes seguir estos pasos:

- En primer lugar, se llamará a `filtrarGastos` para obtener el subconjunto de gastos creados entre las fechas indicadas y que tengan alguna de las etiquetas proporcionadas en el parámetro correspondiente.

- Ejecuta **reduce** sobre el conjunto de gastos filtrados. El valor inicial del acumulador de **reduce** será un **objeto vacío**. Dentro del cuerpo de la función de **reduce**, para cada **gasto** se obtendrá su **periodo de agrupación** (a través del método **obtenerPeriodoAgrupacion** del gasto y el parámetro **periodo** de la función), que se utilizará para **identificar la propiedad del acumulador, sobre la que se sumará su valor**. Así, si **periodo = mes**, un gasto con fecha **2021-11-01** tendrá un periodo de agrupación **2021-11**, por lo que su valor se sumará a **acc["2021-11"]** (siempre que la variable del acumulador haya recibido el nombre **acc** en la llamada a **reduce**). Tienes una pista sobre cómo proceder en la siguiente **pregunta de Stack Overflow**.
- El resultado de **reduce** será el valor de vuelta de la función **agruparGastos**.